



步態辨識門禁系統

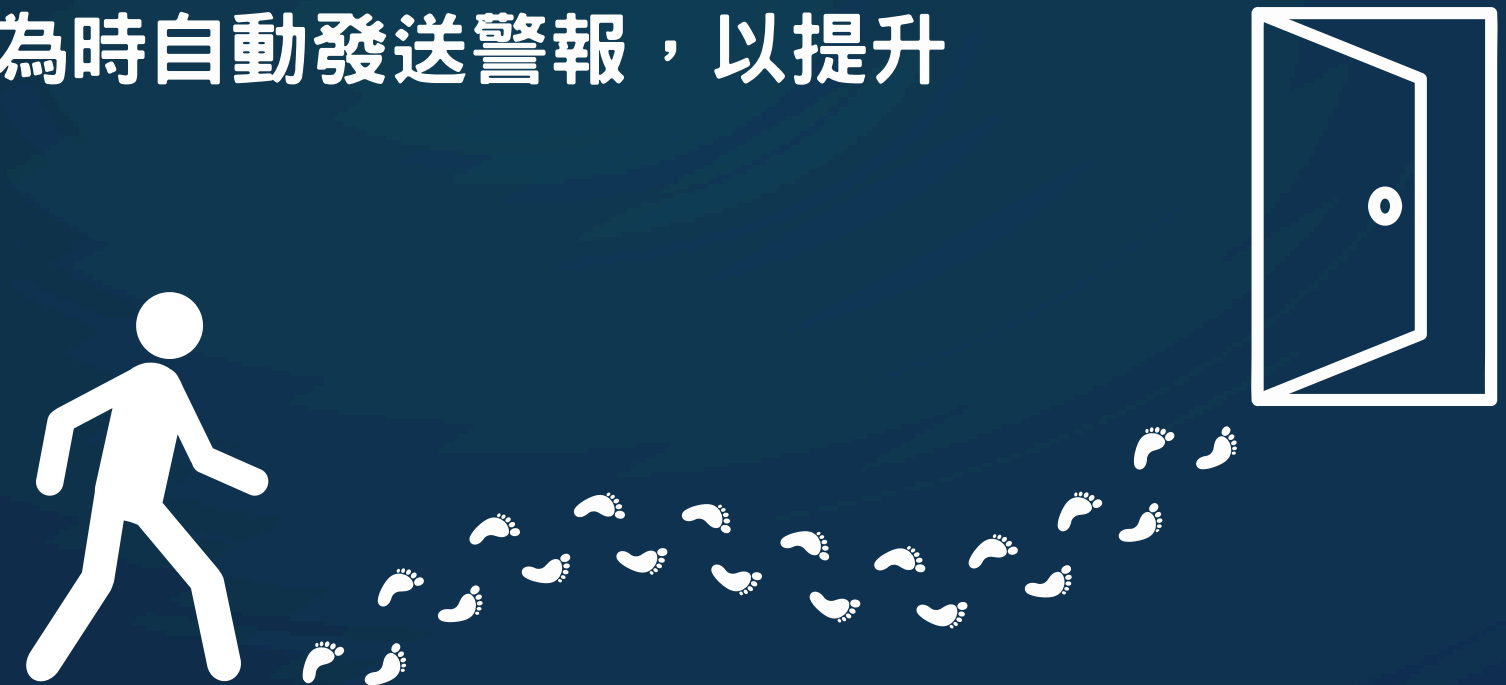
電通所 Q36134255 郭人瑋
統計系 B54106176 古宜庭

一、研究動機

近年來，新聞與網路上經常出現夜間返家時遭陌生人尾隨的事件，尤其女性、兒童與長者在進出住宅或宿舍門禁時，容易因環境昏暗或身旁無人而面臨安全風險。

傳統門禁系統通常只能確認「是否有人刷卡或通過驗證」，但難以判斷授權者後方是否有未授權人員跟隨進入。

因此，我們設計一套結合步態辨識與尾隨偵測的智慧門禁系統，在不依賴清楚臉部影像的情況下辨識授權者，並於偵測到可疑尾隨行為時自動發送警報，以提升居家、宿舍與社區出入口的安全性。



二、系統流程架構



二、系統流程架構



二、系統流程架構

Scenario 1: person_a 正常進門

- 系統辨識為授權者
- 發送「person_a 已抵達」訊息

Scenario 2: unknown 單獨靠近門口

- 系統判斷為 unknown
- 不給予授權

Scenario 3: person_a 通過後 unknown 跟在後面

- 系統偵測 tailgating
- 發送 Discord 警報

三、為什麼我們選 YOLOv8n-pose ?

- Model choice: YOLOv8n-pose
 - YOLOv8-pose 可同時輸出人物框與 17 個人體關鍵點
 - 本專案選擇 YOLOv8n-pose，因為它是最輕量版本
 - 參數量約 3.3M，FLOPs 約 9.2B
 - 較適合 Jetson Xavier 這類 edge device 即時推論
 - 雖然大型模型準確率較高，但推論成本也較高



三、AI Model & 訓練方法

層級	模型/方法	輸出結果
人物偵測	YOLOv8n-pose	人物框 + 17 個人體關鍵點
人物追蹤	Centroid Tracker	每個人的 track_id
步態特徵萃取	Keypoint-based descriptor	21 維步態特徵向量
身份分類	Cosine similarity matching	person_a / unknown
推論加速	TensorRT engine	提升 xavier 上的即時 FPS

本系統的 AI model 分成兩個部分:

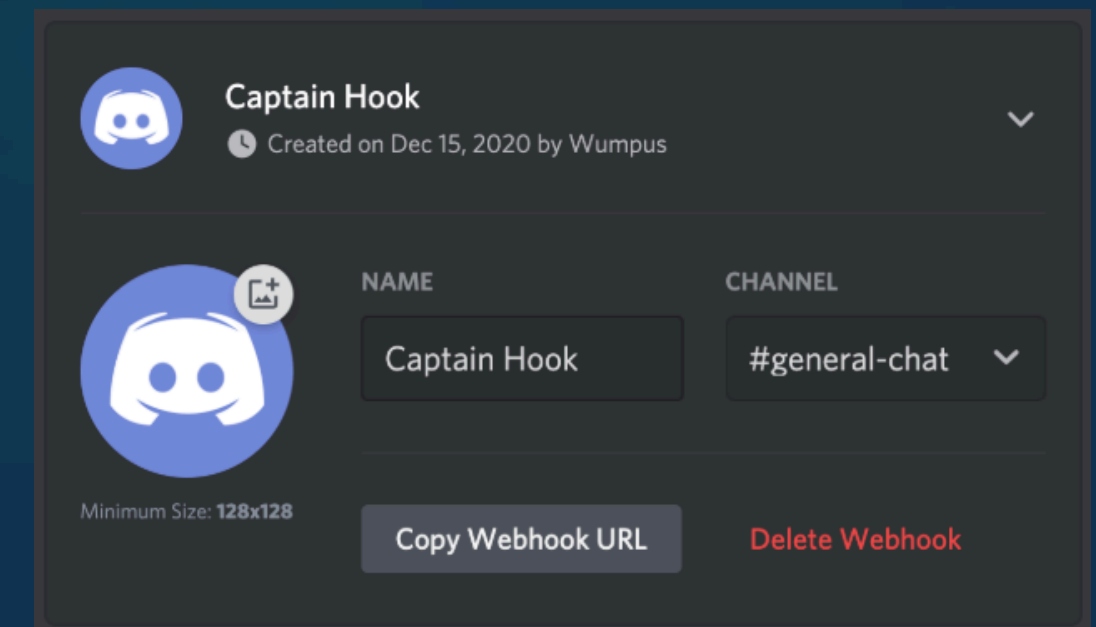
- 第一部分使用 COCO pretrained YOLOv8-pose 偵測人體，取得 bounding box 與 17 個骨架關鍵點
- 第二部分根據同一個人在連續影格中的 keypoint sequence 萃取步態特徵
- 訓練時，不重新訓練 YOLO，而是將 persona 的步態特徵存成授權者模板，並加入 unknown 影片作為負樣本 gallery。
- 推論時，系統會將即時步態特徵與 gallery 進行比對，判斷該人為 person a 或 unknown。

三、Track-level 開放集步態身份驗證演算法設計

自訂模組	核心概念	目的
Gait descriptor	將連續人體 keypoints 轉換成 21 維步態特徵	用走路方式表示身份
Gallery matching	使用 cosine similarity 比對 person_a gallery	判斷是否為授權者
Unknown rejection	透過 unknown gallery 比對負樣本	避免陌生人被誤接受
Track-level voting	累積同一 track 的近期辨識結果	降低單幀抖動
Hysteresis	使用雙門檻進行身份維持	穩定 person_a / unknown 判斷

四、Dashboard 與 Discord 警報串接

- Dashboard:
 - FastAPI 提供 `/state` 與 `/stream.mjpg` API
 - 前端每秒 `fetch /state`
 - 即時顯示 FPS、track label、confidence 與 event log
- Discord:
 - AI pipeline 偵測 `authorized_entry / tailgating event`
 - Event payload 傳給 `AlertNotifier`
 - `AlertNotifier` 讀取 `.env` 的 `ALERT_WEBHOOK_URL`
 - 使用 `HTTP POST` 發送到 Discord webhook



五、實施改善

- 本系統使用 Docker 封裝 dashboard、AI pipeline、模型與警報功能，方便在 Jetson Xavier 上穩定部署。
- Track-level Voting + Hysteresis
 - 因為單幀辨識結果容易因 keypoint 抖動而不穩定
 - 所以加入Track-level voting 累積同一個 track 最近幾次結果
 - Moving average 平滑分數
 - Hysteresis 使用雙門檻：
 - 第一次接受 person_a 使用較高門檻
 - 已確認後使用較低門檻維持身份

五、實施改善 - Jetson Xavier 上的 FPS 優化

- Problem :

一開始使用 ONNXRuntime 執行 YOLOv8-pose，但在 Jetson container 中只偵測到 CPUExecutionProvider，導致推論實際上跑在 CPU 上，Dashboard FPS 約只有 3 - 5 FPS

- Observation :

`onnxruntime.get_available_providers()`

→ ['AzureExecutionProvider', 'CPUExecutionProvider']

- Optimization:

- 降低攝影機解析度，減少每幀影像處理成本
- 將 YOLOv8n-pose ONNX 轉換成 TensorRT engine
- 在 detector.py 加入 TensorRT backend
- Docker image 加入 PyCUDA，處理 CUDA context
- 使用 `--runtime nvidia` 讓 container 存取 Jetson GPU

六、分析結果

指標	結果
整體準確率	87.5%
授權者辨識率	50.0%
陌生人拒絕率	100%

雖然系統在目前測試資料上達到 87.5% 整體準確率，但是 persona 的辨識仍不夠穩定，主要受限於資料量少、視角變化與手刻步態特徵的辨識能力。

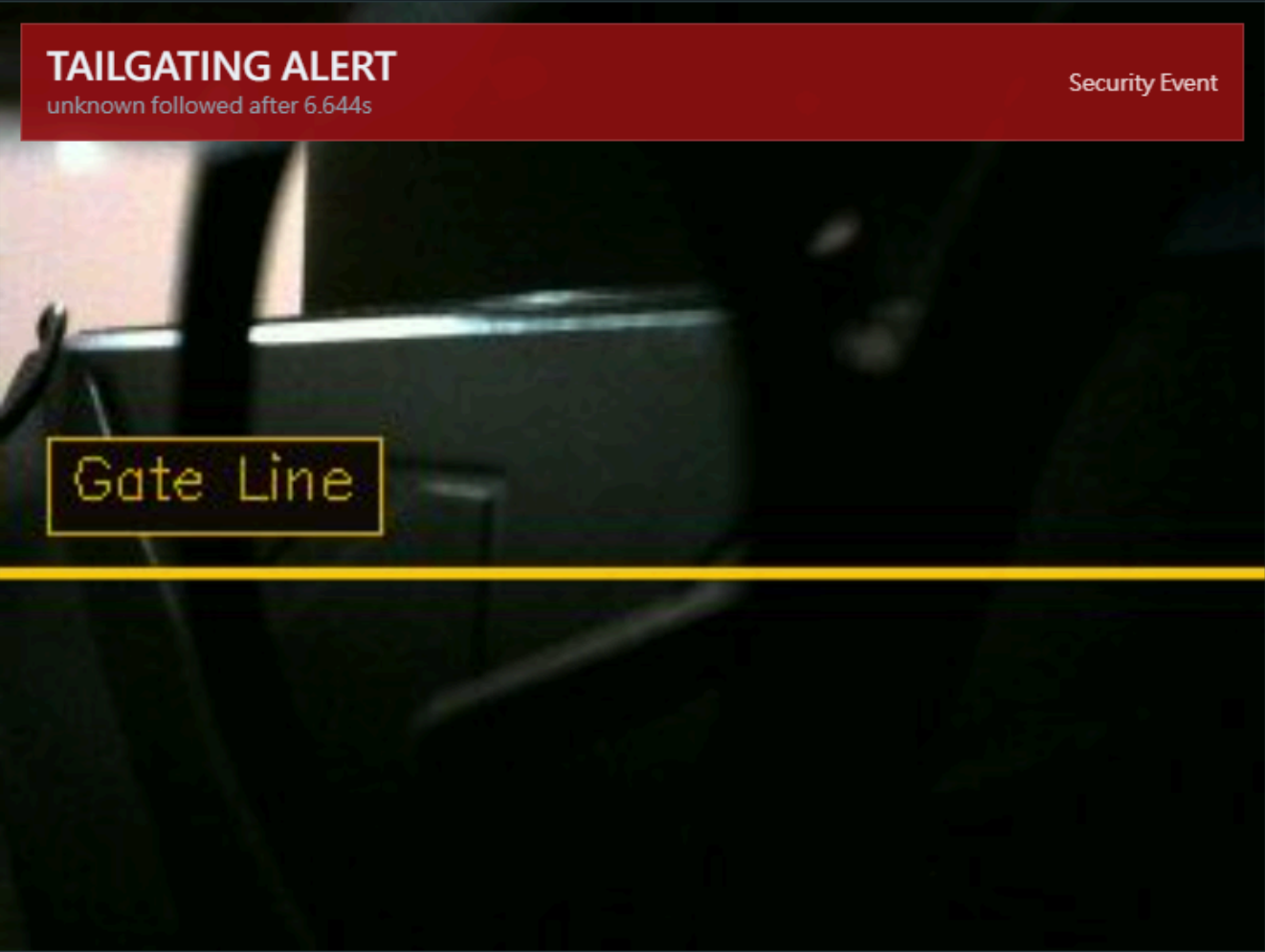
七、UI 介面

GaitPass Jetson

TAILGATING ALERT

unknown followed after 6.644s

Security Event



MonitorEventsDebug

Access State

Tailgating

FPS

14.4

Detector

tensorrt

Alert

cooldown

CURRENT PERSON

No active track

GATE

Gate line60%

Tailgating window10s

Camera0

Frames233083

MonitorEventsDebug

LATEST EVENTS

23:10:38

Tailgating

unknown after 6.644s. Alert cooldown

23:10:36

Tailgating

unknown after 4.483s. Alert queued

23:10:32

Authorized Entry

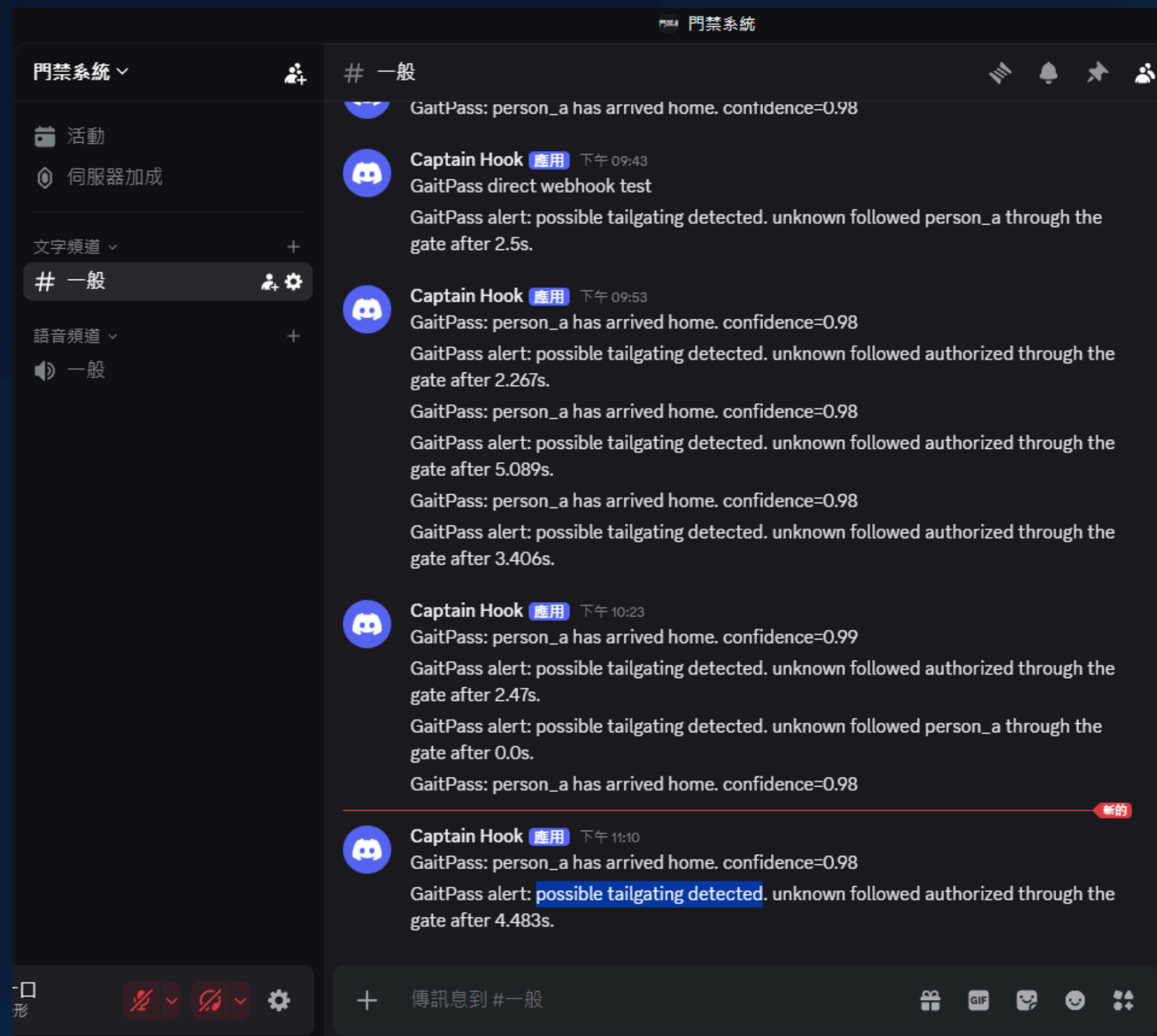
person_a confidence 0.985. Alert queued

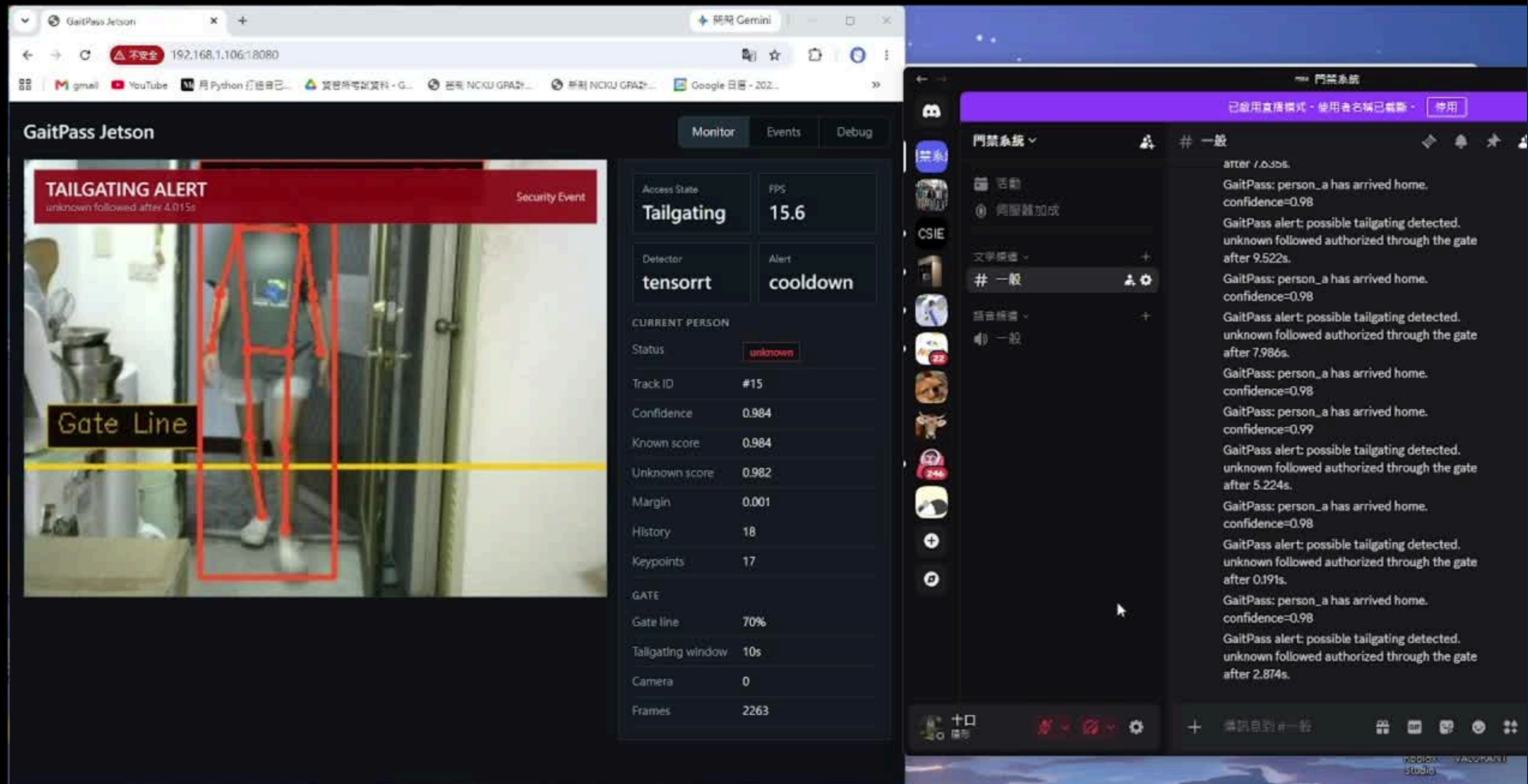
MonitorEventsDebug

RAW STATE

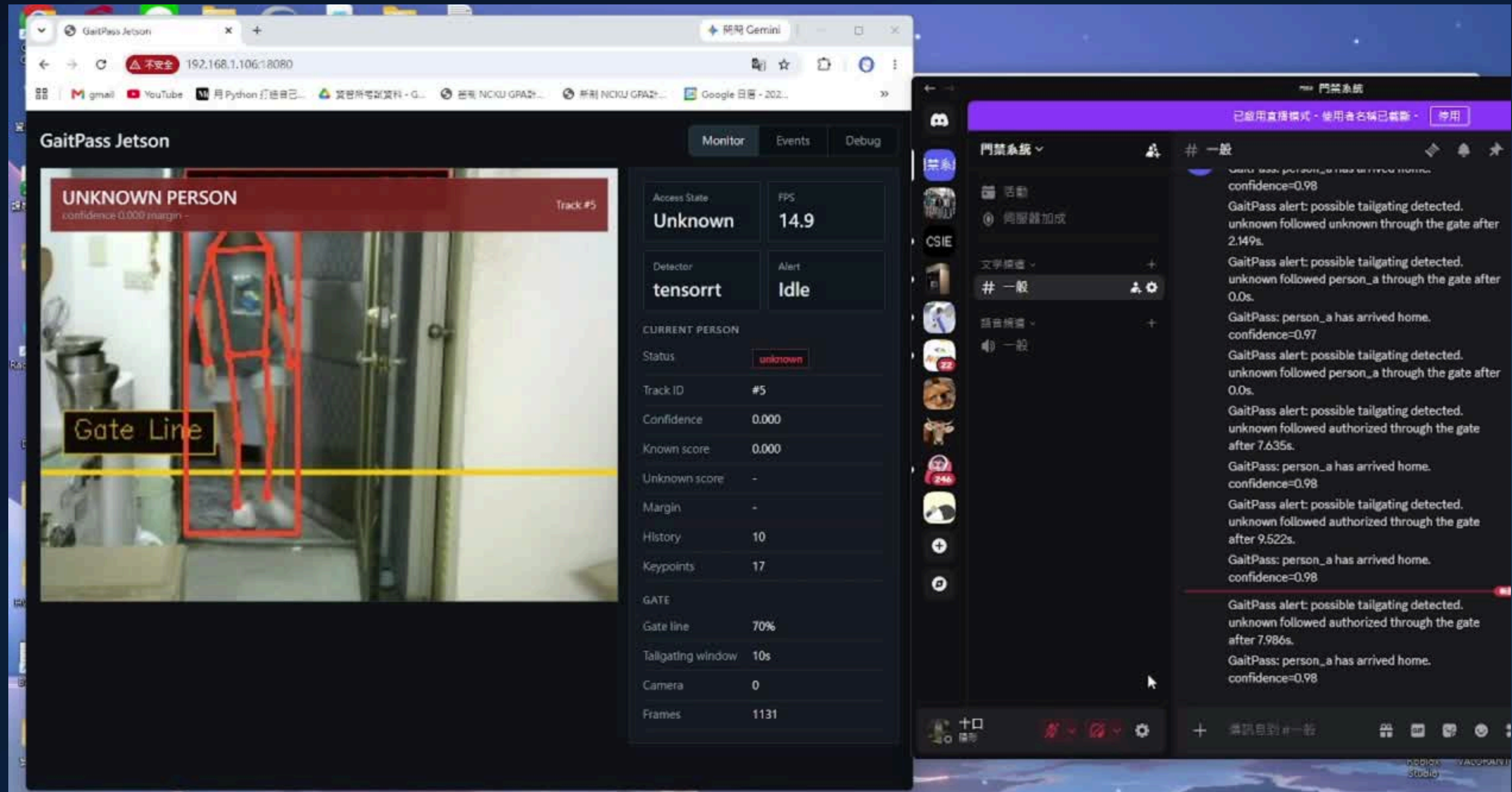
```
{
  "running": true,
  "fps": 14.52,
  "frame_count": 233978,
  "last_error": null,
  "detector": "tensorrt",
  "camera_source": "0",
  "gate_line_y_ratio": 0.6,
  "tailgating_window_sec": 10,
  "tracks": [],
  "events": [
    {
      "ts": "2026-06-11T15:10:38.797247+00",
      "type": "tailgating",
      "authorized_track_id": 3,
      "authorized_label": "authorized",
      "follower_track_id": 4,
      "follower_label": "unknown",
      "time_gap_sec": 6.644,
      "alert": {
        "sent": false,
        "reason": "cooldown"
      }
    },
    {
      "ts": "2026-06-11T15:10:36.636994+00",
      "type": "tailgating",
      "authorized_track_id": 3,
      "authorized_label": "authorized",
      "follower_track_id": 4,
      "follower_label": "unknown",
      "time_gap_sec": 4.483,
      "alert": {
        "sent": true,
        "reason": "queued"
      }
    }
  ]
}
```


七、UI 介面





<https://www.youtube.com/watch?v=xgCrkU1QAww>



https://www.youtube.com/watch?v=AgJBvK4I_RA

八、Conclusion/Future Work

Conclusion:

- 目前系統偏向保守與安全優先。
- 系統能成功拒絕 unknown 使用者，但 person_a 辨識仍不夠穩定。
- 主要挑戰來自資料量少、視角變化，以及手刻步態特徵的限制。
- 對門禁系統而言，誤接受陌生人比誤拒絕授權者更危險。

Future Work:

- 蒐集更多不同使用者的步行影片，提升模型泛化能力。
- 增加不同變化：方向、速度、衣著、背包與光線。
- 針對正面、側面、背面步行建立不同 gallery，降低視角差異造成的誤判。
- 導入 LSTM、GaitGraph 或 OpenGait 等 deep gait embedding model，直接從 keypoint sequence 學習更穩定的步態特徵。
- 改善多人追蹤，降低 ID switching 對尾隨偵測的影響。
- 補充 Jetson 上的 FPS 與 latency benchmark，量化 edge AI 部署效能。

Thank you for your attention